LingoLive — Find your next words

AI Builder Cup 2026 · CreativeEngine · Ashish T. Vasant and Janhvi Singh · 18 October 2026

Theme: Media, Content & Digital Experiences. **Category:** Education.

Problem statement: Help adult language learners understand everyday spoken conversations and formulate a useful next reply, with explanations they can read and learning they can revisit.

Live prototype: https://lingolive-358650584819.us-central1.run.app

Source: https://github.com/U4AR/LingoLive

Current product update — web 1.11

The prototype now builds local personal memory across conversations: useful activities, broad places and preferences, with source evidence and editable confirmation. Inferred routines remain possible; repeated observations improve relevance without becoming facts. Both existing AI branches receive up to five relevant general items and at most two notes from a saved place matched on a return visit, so no third inference call is added. This adapts replies through context, not model-weight fine-tuning.

The primary audience includes immigrants and people moving between Indian states. A Bengaluru/Kannada scenario illustrates the need: someone asks a question while the newcomer needs a useful reply. Live Conversation now prioritizes translation, three suggested replies and pronunciation/playback/copy aids. Word meanings and practice open afterwards in a separate Saved review, using already prepared data without another AI request. The earlier video is archived. The user approved the new Kannada transcript; the original story and approved extension introduce learning near the end as an extra.

The problem

Knowing individual words does not always help someone respond in a real conversation. Immigrants, interstate newcomers and people working in another language may understand part of a sentence but struggle with the next turn. A separate translation screen, pronunciation tool and vocabulary notebook interrupt the exchange. A suggested reply can also be wrong when it forgets what the learner already wanted to do.

LingoLive helps the learner find a useful reply during the exchange, then turns the saved conversation into learning material afterwards. Its intended benefit is more accessible, contextual language practice. Learning gains and real-world accessibility outcomes have not yet been measured.

What the prototype does

The learner chooses a language they understand and one they want to learn. A first-use tutorial introduces live replies, then later word review using a clearly labelled Kannada example. It makes no paid model call and starts no microphone or location request. The main screen then presents one large listening button; typed and pasted phrases are also supported.

Google transcribes speech across languages. Finalized speech becomes a lesson when the selected learning language is present. Other finalized speech remains available as a transcript. Mixed-language phrases can be handled, although language detection is an estimate. Interim captions remain temporary.

The lesson shows the source, a translation and three short suggested replies. After the conversation, opening that phrase in Saved makes source and reply words tappable for their contextual meanings and pronunciation. Original-script and pronunciation views use the same stored word data. Hearing or copying a reply, switching views and opening a word meaning do not require another model call. Voice playback depends on device voices.

Saved lessons can be searched, reopened and deleted on the same browser. The interface keeps settings and detailed explanations behind optional controls. The Android companion exists separately; this submission demonstrates the current deployed web application.

Recognize learner readbacks before generating another reply

Spoken suggestions are checked locally before requesting a new lesson. Current and recent displayed replies, their pronunciation forms, character similarity, word overlap, bigram similarity and substantial partial matches provide redundant ways to recognize a learner reading a reply. Full and partial repeats are saved as learner turns, keep the existing reply cards and make no new generation call. A just-displayed short reply can also match briefly. Negation, changed numbers or a changed question prompt speaker confirmation. Explicit corrections take priority. These are conservative heuristics, not proof of semantic equivalence.

Optional session-only acoustic estimates from microphone frames provide another cue. At most three recent anchors per role expire after three minutes; voice features stay in RAM and never enter saved lessons or Google lesson requests. A recognized other-voice estimate overrides a textual repeat. Fuzzy matches do not train anchors. The learner can correct who spoke or explicitly use Help me say this. There is no paid speaker-classifier call. Twenty-five routing checks, ten real-browser routing checks and 112 backend tests pass. Synthetic held-out voice checks are limited evidence, not noisy-phone speaker-identification accuracy.

Replies that follow the conversation

The learner can state a goal, such as wanting tea to take away. Later replies receive that goal alongside the current utterance, recent turns, the previous suggestions, a short scene summary, learner notes and previously seen vocabulary. When no goal is supplied, the model estimates the immediate purpose and offers plausible replies or clarification. Explicit goals override tentative inferences.

Suggested text is kept separate from confirmed speech. The learner can mark “I said this” to record a reply without another generation request. Model-derived speaker labels are estimates from the conversation, not verified identity or reliable diarization. Seen vocabulary is not presented as mastered vocabulary.

Memory is bounded: at most eight recent turns, twelve seen words, editable notes of 800 characters and a scene that expires after 90 minutes of inactivity. New conversation resets the scene while preserving saved learning and up to 60 personal-memory items. One-off plans expire after a day, possible routines

after 30 days and confirmed routines after 180 days. Settings provide edit, confirmation, forget and pause controls. Approximate coordinates are never turned into a location trail; broad named places come from learner statements or manual entries. Local learner profiles separate notes and history; they are not authenticated accounts, and anyone using the same browser can switch them.

Prepare an intention before the task

The learner can prepare an itinerary at home without an AI request: “Reach the railway station by cab; maybe stop for tea on the way.” One explicit local plan per learner survives page reload, conversation changes and temporary scene expiry for a day. It contains a 600-character intention and at most six ordered steps, with Optional controls, a current step and explicit Done/Skip/Previous actions. The learner can pause, clear or edit it; optional stops are not commitments.

Both existing AI branches receive that plan so the destination and intermediate intent stay available throughout the exchange. Current words and a specific conversation goal override broader plans. Source understanding may draft steps for an unstructured plan in the same request; they require user acceptance. Neither GPS nor AI suggestions mark tasks done or execute transport/booking operations. A future destination is not a current location or a place-memory fact. Plan/revision checks reject late drafts after changes, and plan-scoped old context is excluded after the plan changes. Copied plan payloads are not retained in each saved lesson request snapshot. Expired plans are withheld until edited/resaved.

Help when the learner replies in the wrong language

Previously, known-language-only speech could end at a saved transcript. The new expression path offers three ways to say that intent in the selected learning language, with translation and pronunciation, rather than answering it as though the other person spoke. A learner saying “Could we stop for tea first?” can get a Kannada request. Known-language expression is automatic; **Help me say this** is an explicit correction action. The backend enforces this routing for a detected known-language source even if the model mislabels its response mode. Before the parallel calls, a small deterministic local language detector selects a dedicated meaning-preserving expression prompt for high-confidence known-language text (at least 12 letters, probability at least 0.95). Short or uncertain input keeps model routing; the explicit action always selects expression. There is no third paid classifier call. Role/language detection remains an estimate.

Original language detection remains honest: English input without Kannada has contains_learning_language=false and no invented Kannada source words, while reply_mode=express carries the three actual target phrases. Learning-language input otherwise receives conversational replies. Both flows save target-word meanings for later review and retain the same two-call cost/latency architecture.

Meaningful use of Google AI and Google Cloud

A single Cloud Run service serves the web interface, FastAPI lesson API and authenticated speech WebSocket from one HTTPS origin. Browser AudioWorklet code resamples microphone input to mono 16 kHz PCM. **Gemini 3.5 Transcribe Live** provides streaming transcription. The backend uses **Gemini 3.1**

Flash-Lite with minimal thinking for two parallel tasks: source understanding and reply generation.

Both tasks receive the current transcript and incoming learner/conversation context. The source task returns translation, word meanings, readings and scene estimates. The reply task returns three short replies with translations, meanings and readings. The backend validates and merges both into the existing saved lesson format. Compact nonempty word triples reduce generated output; spacing and punctuation are restored deterministically from the exact source. Missing, changed or reordered lexical content and missing definitions fail validation.

There is no third summarization call or automatic retry. If a branch fails, its sibling is cancelled and the transcript remains available for manual retry. Replies use incoming context immediately; newly inferred source/audio context can influence the next turn, rather than delaying the reply branch.

The host uses Vertex AI with a dedicated Cloud Run service account, without a Google API key in the browser or recommended host runtime. Alternatively, users can connect their own Google AI Studio key. Personal-key mode uses that key for both speech and lessons and never falls back to host credentials.

Guarded place memory and optional setting awareness

Place names previously had no geographic return guard. The current app keeps a separate opt-in store: at most **five places**, **three notes per place**, and **two place notes per AI request**. General preferences remain separate. Saved place notes join context only on a fresh, stationary match after a return visit. The first save does not immediately replay itself. A future destination or an unlinked place name does not establish where the learner currently is.

The source-understanding call may propose one label and fact with an exact quote from the learner's current words. The AI supplies no coordinates and cannot call a location-write tool. The browser checks opt-in, high-confidence/confirmed learner speech, current location, dwell, capacity and an unchanged visit/epoch ticket. Automatic writes are capped at two per rolling day, 30 minutes apart, one per stay. A full store refuses new entries. No third generation or paid Maps call is added.

Place anchors are rounded to three decimal coordinates (about a 100-metre cell), retained only in the local learner profile, with up to three short notes and minimal visit state. Current fine coordinates remain in RAM. A match requires fresh GPS (no older than 90 seconds), reported accuracy at most 75 m, optional speed at most 2.5 m/s, and two distinct samples spanning a stationary minute. The matching rule is **distance + reported accuracy + 80 m rounding allowance <= 250 m**. This recognizes a broad area; it cannot verify an exact business or distinguish nearby shops.

A return means observed departure and stable re-entry, or a fresh stable match following at least 30 minutes without observations. The inactivity alternative supports closing the app between visits but cannot prove physical departure. Poor/missing/stale GPS, movement, a different area, location being off or a different learner withholds the notes. Foreground-only location use stops on hide/profile change and starts off on reload. Only label/fact/status reach Google; saved anchor coordinates do not. Turning on saved places does not enable separate approximate- location sharing. No GPS trail or raw audio archive is created.

Place-specific facts are excluded from general-memory extraction. Older unlinked place notes remain editable but are excluded from automatic retrieval. Derived summaries/history from a recalled place are excluded after a place change, and place-derived tentative intent is cleared. Current explicit goals and statements still take priority. Place notes start as Possible and can be confirmed, edited or forgotten. Place storage expires after 90 days without a note update. Epoch/visit checks block in-flight responses after a forget or pause; original saved conversations have separate deletion controls.

Optional temporary setting signals remain available: a manually chosen scene, current approximate location rounded to two decimals, and up to four seconds of recent sound for the source branch. Those location/audio samples are not saved with lesson history. No Maps service is called.

Performance: measured improvement, with limits

The earlier one-call Gemini 3.8 Flash deployment averaged **9.26 seconds** over ten complete lesson requests. A direct same-model comparison found that simply splitting the original output into two parallel requests was insufficient: three prepared cases averaged 6.71 seconds with one call and 9.55 seconds with two original-format calls. Compact parallel output on 3.8 averaged 3.61 seconds.

The earlier web 1.5 measurement combines parallel requests, Flash-Lite, concise output and warm HTTP connection reuse. It averaged **1.79 seconds over ten complete, validated hosted lessons with zero failures**. The six sequential Japanese/Hindi/Spanish requests averaged 1.75 seconds. Two simultaneous independent sessions completed in 1.65 and 1.71 seconds. A separate functional context/audio check took 2.34 seconds; including it gives an eleven-lesson mean of 1.84 seconds.

Those earlier latency samples predate the new personal-memory prompt. In web 1.7, browser interactions took 2.040 and 7.604 seconds, including click auto-wait. A later mobile test found a status notice could intercept submission; local click-to-request time changed from 6.046 to 0.134 seconds after the UI fix. These browser interaction samples do not isolate Google latency. Separate direct-HTTP memory requests took 1.989 and 2.029 seconds, while two without memory took 1.933 and 1.567 seconds. All six succeeded. The slow call remains in the evidence; two samples per mode do not establish causality or a new ten-request average. Final web 1.8 functional HTTP requests took 2.254, 1.864 and 1.691 seconds (mean 1.936); full browser interactions took 2.335, 1.983 and 1.814 seconds. Request timing starts on the browser network event; interaction timing starts before click. These three simulated-location cases are functional checks, not a ten-request benchmark or noisy physical speech test. These are small warm-path samples, not a promise for every language, long input, mobile network or cold start. The synthesized speech test took 3.54 seconds from the end of audio to the complete lesson, including 1.11 seconds of test-client handoff. Thus the measured two-second lesson average does not describe the whole speech pipeline. Physical-phone latency remains unverified. Slower and failed experiments are retained in the repository rather than excluded from the comparison.

Privacy, authentication and cost controls

Host-funded usage requires a private access code. Browser sessions use signed, opaque, Secure, HttpOnly, SameSite=Strict cookies and expire after one hour. Cookie writes and WebSocket connections require the app's own origin. Google account sign-in is optional in the code but disabled in this deployment.

Personal API keys stay in server RAM for at most one hour. They are not stored in cookies, IndexedDB, LocalStorage, source code or a transcript database. Disconnect revokes the session; restart or scale-to-zero forgets personal keys. Users must trust the host that receives their key. Google bills the personal key's project; the host still pays relay infrastructure costs.

Lessons and vocabulary stay in browser IndexedDB; notes and settings stay in LocalStorage. The app does not create a server-side conversation history database or save raw audio recordings. Selected transcript/context necessarily goes to Google to provide the requested AI features. Clearing browser data deletes local learning. There is no account sync or cross-device history.

Cloud Run has zero minimum and one maximum instance, one CPU, 512 MiB and request-based billing. The backend permits two simultaneous lessons and two active listeners. Raw model-call allowances remain 24/minute, 360/hour and 400/day; two calls per parallel lesson consume two units, allowing 12/180/200 lessons. The two branches share a combined maximum of 4,096 output tokens. Host and personal-key groups have separate shared model allowances. Audio connection time, including silence, is limited to 40 combined minutes/hour and 60/day. Authentication, input bounds, audio pacing and an emergency AI pause run before paid operations. In-memory counters reset after restarts and are usage guards, not a hard dollar cap.

Validation and current readiness

112 backend tests, **25 speech-routing checks**, **10 routing browser checks**, plus the existing general, audio, memory, plan and guarded-place checks passed, along with JavaScript source/word/audio/context checks. Local tests use explicit Google, microphone and location fixtures. Separate hosted checks used real Google transcription and lessons, validated exact text/readings/meanings, checked access controls and proved invalid personal keys do not produce a host-funded fallback. Valid personal-key inference and physical-phone capture remain unverified.

Production deployment details in `validation/cloud-current-deployment.json` identify web 1.11 on Cloud Run (`lingolive-00029-mec`). The same revision was checked through a temporary tag before promotion; public health, assets and routing were verified afterward. The normal persistent Cloud SDK operator login is separate from the runtime service account. No deployment credentials are included in the repository or demonstration.

Final web 1.9 typed functional requests took 2.196, 1.894, 1.914, 2.255 seconds (mean 2.065); browser interactions took 2.340, 1.939, 1.972, 2.307 seconds. All four cases passed: Kannada cab directions, an English tea request expressed in Kannada, a Kannada café exchange with explicit plan progress, and declining tea to go to the station. These are four functional cases, not a stable latency average, physical speech capture test or evidence of sub-two-second performance across users. Three earlier staged failures and direct diagnostics remain in the evidence; they are not included in this final passing sample.

Current video demonstration

The current submission video runs approximately **3:17**, within the supplied 3–4-minute requirement. The original approved Bengaluru/Kannada story presents the communication problem before the solution. An approved extension demonstrates real full and partial readbacks against deployed web 1.11. Learning

remains an additional feature near the end, at **2:52**. The export is 1080p H.264/AAC with English captions and no artificial noise bed. Google Gemini 3.1 Flash TTS preview provides the original natural-speed voices. No audio is time-stretched.

The extension uses actual Google transcription and lessons. Both learner readbacks made zero extra lesson requests; the next driver's speech made one lesson request carrying both actual readback transcripts as learner context. The capture's original exact-string assertion failed because ASR added a full stop to the intended TTS script. Independent verification compares the actual recognized transcript with the subsequent context; both turns pass. The original failure is retained rather than relabelled as a clean capture run.

The footage shows genuine ASR limitations. The original café question was misidentified as Bengali and visibly corrected. The extension's Kannada driver question appeared in Telugu script with an incorrect station name; actual app output remains visible with a notice. Replies still used the Bengaluru trip context. Model outputs are not replaced. Generated opening art, synthetic voices, microphone input and simulated location/time are disclosed. No physical microphone or native-human language review is claimed.

The original seven final-take lesson requests and three earlier incomplete-take requests remain documented. The extension added two lesson requests plus cached actor/narration synthesis. Final audio verification compares each expected source waveform with the decoded MP4, checks speech level, clipping and silent gaps. No new model calls are needed to render, verify or replay the submission video.

Differentiation and growth

The prototype brings together live conversation, contextual next-turn replies, tappable words, pronunciation and local learning memory in one small interface. Its engineering focus is a responsive complete lesson, rather than requiring a new paid call for each word or pronunciation toggle. It does not claim to be the first translation or conversational tutoring product.

For wider adoption, the next steps are representative learner studies, mobile and accessibility testing, language-quality evaluation, durable per-user quotas, stronger identity and a deliberate design for cross-instance key sessions. The current maximum-one-instance setting is a prototype cost choice, not a production scaling claim. Learning retention, task completion, reply usefulness, word-meaning accuracy and real-device latency should be measured before claiming educational outcomes.

Submission materials and references

- Official-template pitch: `LingoLive-Pitch-Deck.pdf` and editable `LingoLive-Pitch-Deck.pptx` in this directory.
- Prepared form responses: `ENTRY.md` and `submission.json`.
- Current demo: `DEMO-TRANSCRIPT-V2.md`, approved `DEMO-EXTENSION-APPROVED.md` and `validation/submission-demo-current.json`. Earlier exports remain archived.
- Technical detail: `MEMORY.md`, `WEB-DEPLOY.md`, `VALIDATION.md` and `docs/LATENCY.md`.
- Official requirements and supplied template checked October 18, 2026: https://aibuildercup.com/themes.html and https://aibuildercup.com/Faqs.html.

- Registered participants access their dashboard at https://hack2skill.com/event/aibuildercup2026/registration?isExternalLinkRedirection=true.

The supplied requirements mandate the official pitch template, a 3–4-minute functional video, a deployed prototype, accessible source and a brief description. The official terms require public source. The registered team screenshot identifies CreativeEngine, Ashish T. Vasant (leader) and Janhvi Singh. The public site lists October 18 as the deadline; exact time zone/cutoff and portal declarations must be checked at submission. No portal submission or receipt is claimed. A public MP4 is provided; if the portal demands a particular host, upload it to the accepted service before entering that URL.

Firebase and Firestore are **not used** in this version. Browser IndexedDB and LocalStorage hold local learning/memory; Cloud Run hosts the application and Google AI relay. Firestore could support future opt-in synchronization, subject to separate privacy and account design. No cross-device sync is claimed.